Mobile Application Development

LAB MANUAL

MCA Semester III | Course Code: MCA03EL03CL

Practical: 2 hrs/week | TW 10, PR 30, OR 10

Name: _____ Roll No.: _____ Batch: _____

Course Outcomes

- CO1: Design the right user interface for a mobile application.
- CO2: Implement a mobile application using UI toolkits and frameworks.
- CO3: Design mobile applications that are aware of the resource constraints of mobile devices.
- CO4: Develop web-based mobile applications that access internet and location data.

Index

Sr.	Practical	Syllabus Assignment	Date	Sign
1	Android Studio Setup and Hello World	1		
2	Activity Life Cycle with Logcat	2		
3	User Registration Form using Linear Layout	3		
4	Calculator UI using Grid Layout and Linear Layout	4		
5	Externalizing Resources and Toast Message	5		
6	Two Fragments in Portrait and Landscape	7		
7	Fragment to Activity Communication	8		
8	Explicit and Implicit Intents	9		
9	SQLite Database: Create and Insert	11		
10	SQLite Database: Retrieve with Cursor and ListView	12		

Note: the syllabus lists 12 assignments (any 10 to be completed). This manual covers assignments 1-5, 7-9, 11 and 12 so that all four units are covered. Assignments 6 (options menu) and 10 (startActivityForResult) are optional extras.

General requirements: Android Studio, JDK 17 or later, Android SDK, one AVD (e.g. Pixel with API 30+). Package name used in all programs: com.example.lab.

Practical 1: Android Studio Setup and Hello World

Aim: Install Android Studio, set up the JDK, create a new project and run the default Hello World app on an emulator (AVD).

Objectives:

- To install and configure the JDK, Android SDK and Android Studio.
- To understand the structure of an Android project.
- To create and run an AVD and execute the first application.

Theory: Android Studio is the official IDE for Android. It bundles the Android SDK, Gradle build system, code editor, Logcat and the AVD Manager. An AVD (Android Virtual Device) is an emulated device defined by a hardware profile and a system image. A project contains manifests/, java/, res/ and Gradle scripts; building produces an APK that is installed on the AVD.

Steps:

1. Install the JDK and set the JAVA_HOME environment variable.
2. Download and install Android Studio; complete the setup wizard (SDK, emulator, platform tools).
3. Choose File > New > New Project > Empty Views Activity; set name HelloWorld, package com.example.lab, language Java.
4. Open Tools > Device Manager, create a virtual device and download a system image.
5. Press Run. The app installs on the AVD and displays the text.

Code (activity_main.xml):

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        android:textSize="24sp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Code (MainActivity.java):

```
package com.example.lab;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Expected Output: The emulator shows a screen with the text "Hello World!".

Output Screenshot:

Paste screenshot here
(emulator screen showing Hello World)

Conclusion: Thus, Android Studio and an AVD were set up successfully and the Hello World application was built and run, which confirms the complete workflow from source code to APK.

Viva Questions: What is an AVD? What is the role of Gradle? What is an APK? What is the purpose of setContentView()?

Practical 2: Activity Life Cycle with Logcat

Aim: Create an application with a single activity, override all life cycle methods and use Logcat to observe state changes.

Objectives:

- To understand the states of an Activity and the order of life cycle callbacks.
- To use Log.d() and Logcat to monitor state changes.
- To observe the effect of rotation, Home and Back actions.

Theory: An Activity moves through Created, Started, Resumed, Paused, Stopped and Destroyed states. The system calls onCreate(), onStart(), onResume(), onPause(), onStop(), onRestart() and onDestroy() as the user interacts with the app. Log.d(tag, message) writes debug messages to Logcat.

Steps:

1. Create a project named LifecycleDemo.
2. Override every life cycle method and log its name.
3. Run the app and open Logcat; filter using the tag LIFECYCLE.
4. Rotate the screen, press Home, reopen the app and press Back; note the order of calls.

Code (MainActivity.java):

```
package com.example.lab;

import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    private static final String TAG = "LIFECYCLE";

    @Override protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate");
    }
    @Override protected void onStart()   { super.onStart();   Log.d(TAG, "onStart"); }
    @Override protected void onResume()  { super.onResume();  Log.d(TAG, "onResume");
}
    @Override protected void onPause()   { super.onPause();   Log.d(TAG, "onPause"); }
    @Override protected void onStop()    { super.onStop();    Log.d(TAG, "onStop"); }
    @Override protected void onRestart() { super.onRestart(); Log.d(TAG, "onRestart");
}
    @Override protected void onDestroy() { super.onDestroy(); Log.d(TAG, "onDestroy");
}
}
```

Expected Output: Logcat shows the following sequences:

Action	Methods logged
App launch	onCreate, onStart, onResume
Press Home	onPause, onStop
Reopen app	onRestart, onStart, onResume
Rotate screen	onPause, onStop, onDestroy, onCreate, onStart, onResume
Press Back	onPause, onStop, onDestroy

Output Screenshot:

Paste screenshot here
(Logcat window filtered on LIFECYCLE)

Conclusion: Thus, the life cycle callbacks were overridden and their order was verified in Logcat for launch, Home, return, rotation and Back, which shows how an Activity moves between its states.

Viva Questions: Why is the Activity recreated on rotation? Difference between onPause() and onStop()? When is onSaveInstanceState() called?

Practical 3: User Registration Form using Linear Layout

Aim: Design a registration form using vertical and horizontal Linear Layouts with EditText, TextView, Button and CheckBox.

Objectives:

- To design a user interface using vertical and horizontal Linear Layouts.
- To use EditText, CheckBox, TextView and Button widgets.
- To share space between views using layout_weight.

Theory: LinearLayout arranges children in a single row or column set by android:orientation. Nesting a horizontal layout inside a vertical one allows grouped fields such as first and last name side by side. layout_weight divides remaining space.

Steps:

1. Create project RegistrationForm.
2. Replace activity_main.xml with the layout below.
3. Run on the AVD and verify the layout in portrait and landscape.

Code (activity_main.xml):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="User Registration"
        android:textSize="22sp"
        android:textStyle="bold" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal">
        <EditText
            android:id="@+id/etFirst"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="First name" />
        <EditText
            android:id="@+id/etLast"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="Last name" />
    </LinearLayout>

    <EditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email"
        android:inputType="textEmailAddress" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
```

```
        android:inputType="textPassword" />

    <CheckBox
        android:id="@+id/cbTerms"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="I accept the terms and conditions" />

    <Button
        android:id="@+id/btnRegister"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Register" />
</LinearLayout>
```

Expected Output: A form with two side-by-side name fields, email, password, a checkbox and a Register button.

Output Screenshot:

Paste screenshot here
(registration form on the emulator)

Conclusion: Thus, a registration form was designed using nested Linear Layouts and basic widgets, and the weights divided the screen space equally in both portrait and landscape.

Viva Questions: What does layout_weight do? Why use 0dp width with weights? What are dp and sp?

Practical 4: Calculator UI using Grid Layout and Linear Layout

Aim: Replicate a simple calculator interface using a TextView display and a GridLayout of buttons inside a LinearLayout.

Objectives:

- To build a calculator interface using GridLayout inside a LinearLayout.
- To distribute rows and columns equally using weights.
- To compare Grid, Linear and Table layouts.

Theory: GridLayout places views in rows and columns using rowCount and columnCount. Each button uses layout_columnWeight and layout_rowWeight to share space equally. A vertical LinearLayout stacks the display above the grid.

Steps:

1. Create project CalculatorUI.
2. Add the layout below; the 16 buttons follow the same pattern (7 8 9 /, 4 5 6 *, 1 2 3 -, C 0 = +).
3. Optionally add click logic to evaluate simple expressions.

Code (activity_main.xml (abridged: repeat the Button element for each key)):

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tvDisplay"
        android:layout_width="match_parent"
        android:layout_height="120dp"
        android:gravity="end|center_vertical"
        android:padding="16dp"
        android:text="0"
        android:textSize="36sp" />

    <GridLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:columnCount="4"
        android:rowCount="4">

        <Button android:text="7" android:layout_width="0dp"
android:layout_height="0dp"
            android:layout_columnWeight="1" android:layout_rowWeight="1" />
        <Button android:text="8" android:layout_width="0dp"
android:layout_height="0dp"
            android:layout_columnWeight="1" android:layout_rowWeight="1" />
        <Button android:text="9" android:layout_width="0dp"
android:layout_height="0dp"
            android:layout_columnWeight="1" android:layout_rowWeight="1" />
        <Button android:text="/" android:layout_width="0dp"
android:layout_height="0dp"
            android:layout_columnWeight="1" android:layout_rowWeight="1" />
        <!-- repeat for the remaining 12 buttons -->
    </GridLayout>
</LinearLayout>
```

Expected Output: A calculator screen with a right-aligned display and a 4 x 4 key grid filling the screen.

Output Screenshot:

Paste screenshot here
(calculator screen on the emulator)

Conclusion: Thus, the calculator interface was created with a GridLayout of keys below a display, showing how rows and columns can be aligned and stretched evenly.

Viva Questions: Difference between LinearLayout and GridLayout? What do columnWeight and rowWeight control? When would TableLayout be preferred?

Practical 5: Externalizing Resources and Toast Message

Aim: Modify the registration form to use strings.xml and colors.xml and display a "Welcome, [Username]!" Toast when Register is clicked.

Objectives:

- To keep text and colours in resource files instead of hard-coding them.
- To access resources from XML and Java.
- To give feedback to the user with a Toast message.

Theory: Externalizing resources keeps text and colours out of layouts and code, which supports localization and consistent theming. Resources are referenced as @string/name and @color/name in XML, and R.string.name in Java. A Toast shows a brief message that disappears automatically.

Steps:

1. Open res/values/strings.xml and colors.xml and add the entries below.
2. In the layout, replace literal text with @string/... and colours with @color/....
3. Add the click listener in MainActivity.

Code (res/values/strings.xml):

```
<resources>
    <string name="app_name">RegistrationForm</string>
    <string name="title">User Registration</string>
    <string name="hint_first">First name</string>
    <string name="hint_last">Last name</string>
    <string name="btn_register">Register</string>
    <string name="welcome">Welcome, %1$s!</string>
</resources>
```

Code (res/values/colors.xml):

```
<resources>
    <color name="primary">#1F3A5F</color>
    <color name="accent">#1B998B</color>
    <color name="white">#FFFFFF</color>
</resources>
```

Code (Layout usage):

```
<Button
    android:id="@+id/btnRegister"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/btn_register"
    android:backgroundTint="@color/primary"
    android:textColor="@color/white" />
```

Code (MainActivity.java):

```
package com.example.lab;

import android.os.Bundle;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        EditText etFirst = findViewById(R.id.etFirst);
```

```
        Button btn = findViewById(R.id.btnRegister);

        btn.setOnClickListener(v -> {
            String name = etFirst.getText().toString().trim();
            if (name.isEmpty()) {
                Toast.makeText(this, "Please enter your name",
Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this, getString(R.string.welcome, name),
                        Toast.LENGTH_LONG).show();
            }
        });
    }
}
```

Expected Output: Tapping Register with the name "Rutuja" shows the Toast "Welcome, Rutuja!".

Output Screenshot:

Paste screenshot here
(screen showing the Welcome Toast)

Conclusion: Thus, strings and colours were externalized in strings.xml and colors.xml and a personalised Toast was shown, which makes the application easier to maintain and localise.

Viva Questions: Why externalize strings? What is R.java? Difference between Toast.LENGTH_SHORT and LENGTH_LONG?

Practical 6: Two Fragments in Portrait and Landscape

Aim: Create an app with a list fragment and a detail fragment. Portrait mode shows only the list; landscape mode shows both side by side.

Objectives:

- To create fragments with their own layouts.
- To use alternate layouts (layout and layout-land) for different orientations.
- To understand how fragments are hosted by an Activity.

Theory: A Fragment is a reusable portion of UI with its own layout and life cycle, hosted by an Activity. Keeping separate layout files in res/layout (portrait) and res/layout-land (landscape) lets the same Activity show one or two panes.

Steps:

1. Create project FragmentDemo.
2. Create ListFragment1 and DetailFragment classes with layouts fragment_list.xml and fragment_detail.xml.
3. Create res/layout/activity_main.xml (list only) and res/layout-land/activity_main.xml (list + detail).
4. Run the app and rotate the emulator.

Code (res/layout/activity_main.xml (portrait)):

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <fragment
        android:id="@+id/listFragment"
        android:name="com.example.lab.ListFragment1"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</FrameLayout>
```

Code (res/layout-land/activity_main.xml (landscape)):

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal">
    <fragment
        android:id="@+id/listFragment"
        android:name="com.example.lab.ListFragment1"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />
    <fragment
        android:id="@+id/detailFragment"
        android:name="com.example.lab.DetailFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="2" />
</LinearLayout>
```

Code (ListFragment1.java):

```
package com.example.lab;

import android.os.Bundle;
import android.view.*;
import android.widget.*;
import androidx.annotation.*;
import androidx.fragment.app.Fragment;
```

```
public class ListFragment1 extends Fragment {
    private final String[] items = {"Android", "Fragments", "Intents", "SQLite"};

    @Nullable @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container, @Nullable Bundle state) {
        View v = inflater.inflate(R.layout.fragment_list, container, false);
        ListView lv = v.findViewById(R.id.listView);
        lv.setAdapter(new ArrayAdapter<>(requireContext(),
                android.R.layout.simple_list_item_1, items));
        return v;
    }
}
```

Code (DetailFragment.java):

```
package com.example.lab;

import android.os.Bundle;
import android.view.*;
import androidx.annotation.*;
import androidx.fragment.app.Fragment;

public class DetailFragment extends Fragment {
    @Nullable @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container, @Nullable Bundle state) {
        return inflater.inflate(R.layout.fragment_detail, container, false);
    }
}
```

fragment_list.xml contains a single ListView with id listView; fragment_detail.xml contains a TextView with id tvDetail.

Expected Output: Portrait shows the list only; landscape shows the list on the left and the detail pane on the right.

Output Screenshot:

Paste screenshot here
(portrait and landscape screenshots)

Conclusion: Thus, two fragments were created and hosted in one Activity, and the same app showed a single pane in portrait and two panes in landscape using alternate layouts.

Viva Questions: What is a Fragment? Why is layout-land used? Why must a Fragment be hosted in an Activity?

Practical 7: Fragment to Activity Communication

Aim: Extend Practical 6 so that selecting a list item notifies the hosting Activity through an interface, and the Activity updates the detail fragment.

Objectives:

- To communicate from a Fragment to its host Activity using an interface.
- To update one fragment from the Activity based on an event in another.
- To understand why fragments should not talk to each other directly.

Theory: Fragments should not talk directly to each other. The standard pattern is an interface declared inside the fragment, implemented by the Activity, and called when an event occurs. The Activity then forwards data to the other fragment through a public method.

Steps:

1. Add the OnItemSelectedListener interface to ListFragment1.
2. Attach the listener in onAttach() and call it from the list item click.
3. Implement the interface in MainActivity and call DetailFragment.updateText().

Code (ListFragment1.java (additions)):

```
public interface OnItemSelectedListener {
    void onItemSelected(String item);
}

private OnItemSelectedListener listener;

@Override
public void onAttach(@NonNull android.content.Context context) {
    super.onAttach(context);
    if (context instanceof OnItemSelectedListener) {
        listener = (OnItemSelectedListener) context;
    } else {
        throw new ClassCastException(context + " must implement
OnItemSelectedListener");
    }
}

// inside onCreateView, after setAdapter:
lv.setOnItemClickListener((parent, view, position, id) ->
        listener.onItemSelected(items[position]));
```

Code (DetailFragment.java (addition)):

```
public void updateText(String text) {
    TextView tv = getView().findViewById(R.id.tvDetail);
    tv.setText("You selected: " + text);
}
```

Code (MainActivity.java):

```
package com.example.lab;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity
        implements ListFragment1.OnItemSelectedListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```
    }

    @Override
    public void onItemSelected(String item) {
        DetailFragment detail = (DetailFragment)
                getSupportFragmentManager().findFragmentById(R.id.detailFragment);
        if (detail != null) {
            detail.updateText(item);   // landscape: both panes visible
        }
    }
}
```

Expected Output: In landscape, tapping an item updates the detail pane with "You selected: <item>".

Output Screenshot:

Paste screenshot here
(landscape screen with selected item in the detail pane)

Conclusion: Thus, a callback interface was used to pass the selected item from the list fragment to the Activity, which updated the detail fragment, keeping the fragments reusable and independent.

Viva Questions: Why use an interface instead of direct fragment calls? What does onAttach() do? How would you handle portrait mode (hint: start a second Activity or replace the fragment)?

Practical 8: Explicit and Implicit Intents

Aim: Create two activities, navigate from the first to the second with an explicit Intent passing a string, and open a web page from the second using an implicit Intent.

Objectives:

- To navigate between activities using an explicit Intent.
- To pass data between activities using Intent extras.
- To launch another application (browser) using an implicit Intent.

Theory: An Intent is a messaging object used to request an action. An explicit Intent names the target component class; an implicit Intent declares an action and data, and the system selects a matching app. Data is passed with putExtra() and read with getIntent().getStringExtra(). Every Activity must be declared in AndroidManifest.xml.

Steps:

1. Create project IntentDemo with MainActivity and SecondActivity.
2. Add SecondActivity to the manifest.
3. Add a Button to each layout (btnNext in the first, btnOpen and tvMessage in the second).
4. Run and test both navigation paths.

Code (AndroidManifest.xml (inside application)):

```
<activity android:name=".SecondActivity" android:exported="false" />
```

Code (MainActivity.java):

```
package com.example.lab;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Button next = findViewById(R.id.btnNext);
        next.setOnClickListener(v -> {
            Intent i = new Intent(MainActivity.this, SecondActivity.class); //
explicit
            i.putExtra("msg", "From Main Activity");
            startActivity(i);
        });
    }
}
```

Code (SecondActivity.java):

```
package com.example.lab;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);

        TextView tv = findViewById(R.id.tvMessage);
        tv.setText(getIntent().getStringExtra("msg"));

        Button open = findViewById(R.id.btnOpen);
        open.setOnClickListener(v -> {
            Intent web = new Intent(Intent.ACTION_VIEW,
                    Uri.parse("https://www.google.com"));   // implicit
            startActivity(web);
        });
    }
}
```

Expected Output: The second screen shows "From Main Activity"; tapping Open launches the browser at the given URL.

Output Screenshot:

Paste screenshot here
(second activity and the opened browser)

Conclusion: Thus, an explicit Intent moved the user to the second screen with data, and an implicit Intent opened a web page, showing how components communicate through Intents.

Viva Questions: Explicit vs implicit Intent? What is an Intent extra? What are the main parts of an Intent filter?

Practical 9: SQLite Database: Create and Insert

Aim: Create a form to enter student details (name, roll number). Use a SQLiteOpenHelper class to create a database and a students table and insert the data on button click.

Objectives:

- To create a SQLite database and table using SQLiteOpenHelper.
- To insert records using ContentValues.
- To validate user input before storing it.

Theory: SQLite is an embedded relational database stored privately for each app. SQLiteOpenHelper manages creation and versioning: onCreate() runs the first time the database is created and onUpgrade() runs when the version number increases. ContentValues holds column-value pairs for insert().

Steps:

1. Create project StudentDB with a layout containing etName, etRoll (numeric) and btnSave.
2. Add the DBHelper class.
3. Insert the record in the Save button handler and show a Toast with the result.

Code (DBHelper.java):

```
package com.example.lab;

import android.content.*;
import android.database.sqlite.*;

public class DBHelper extends SQLiteOpenHelper {
    private static final String DB_NAME = "college.db";
    private static final int DB_VERSION = 1;
    public static final String TABLE = "students";

    public DBHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE " + TABLE + "("
                + "id INTEGER PRIMARY KEY AUTOINCREMENT,"
                + "name TEXT NOT NULL,"
                + "roll INTEGER NOT NULL)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE);
        onCreate(db);
    }

    public boolean insertStudent(String name, int roll) {
        SQLiteDatabase db = getWritableDatabase();
        ContentValues cv = new ContentValues();
        cv.put("name", name);
        cv.put("roll", roll);
        return db.insert(TABLE, null, cv) != -1;
    }
}
```

Code (MainActivity.java):

```
package com.example.lab;
```

```
import android.os.Bundle;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    private DBHelper helper;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        helper = new DBHelper(this);

        EditText etName = findViewById(R.id.etName);
        EditText etRoll = findViewById(R.id.etRoll);
        Button save = findViewById(R.id.btnSave);

        save.setOnClickListener(v -> {
            String name = etName.getText().toString().trim();
            String roll = etRoll.getText().toString().trim();
            if (name.isEmpty() || roll.isEmpty()) {
                Toast.makeText(this, "Enter all details", Toast.LENGTH_SHORT).show();
                return;
            }
            boolean ok = helper.insertStudent(name, Integer.parseInt(roll));
            Toast.makeText(this, ok ? "Student saved" : "Insert failed",
                    Toast.LENGTH_SHORT).show();
            etName.setText("");
            etRoll.setText("");
        });
    }
}
```

Expected Output: Saving a valid record shows "Student saved". The database file college.db is created in the app's private storage (viewable in Android Studio's Device Explorer or App Inspection).

Output Screenshot:

Paste screenshot here
(form with the Student saved message)

Conclusion: Thus, a SQLite database and a students table were created and records were inserted successfully, showing how an Android app stores structured data locally.

Viva Questions: When is onCreate() of SQLiteOpenHelper called? What is ContentValues? What does insert() return on failure?

Practical 10: SQLite Database: Retrieve with Cursor and ListView

Aim: Extend Practical 9 to retrieve all student records from SQLite using a Cursor and display them in a ListView.

Objectives:

- To read records from a SQLite table using a Cursor.
- To display the data in a ListView using an adapter.
- To close cursors properly and handle an empty result.

Theory: A Cursor is a pointer over the rows returned by a query. rawQuery() or query() returns it positioned before the first row; moveToNext() advances row by row and getString() / getInt() read columns by index. The cursor must be closed after use. An ArrayAdapter binds a list of strings to a ListView.

Steps:

1. Add a ListView with id lvStudents and a Button btnShow to the layout.
2. Add the getAllStudents() method to DBHelper.
3. Fill the ListView in the Show button handler.

Code (DBHelper.java (addition; also import java.util.*; and android.database.Cursor)):

```
public List<String> getAllStudents() {
    List<String> list = new ArrayList<>();
    SQLiteDatabase db = getReadableDatabase();
    Cursor c = db.rawQuery("SELECT id, name, roll FROM " + TABLE
            + " ORDER BY roll", null);
    while (c.moveToNext()) {
        int id = c.getInt(c.getColumnIndexOrThrow("id"));
        String name = c.getString(c.getColumnIndexOrThrow("name"));
        int roll = c.getInt(c.getColumnIndexOrThrow("roll"));
        list.add("Roll " + roll + " - " + name + " (id " + id + ")");
    }
    c.close();
    return list;
}
```

Code (MainActivity.java (addition inside onCreate; also import java.util.List;)):

```
ListView lv = findViewById(R.id.lvStudents);
Button show = findViewById(R.id.btnShow);

show.setOnClickListener(v -> {
    List<String> students = helper.getAllStudents();
    if (students.isEmpty()) {
        Toast.makeText(this, "No records found", Toast.LENGTH_SHORT).show();
    }
    lv.setAdapter(new ArrayAdapter<>(this,
            android.R.layout.simple_list_item_1, students));
});
```

Expected Output: Tapping Show lists every stored student, for example "Roll 12 - Anjali Narkhede (id 1)".

Output Screenshot:

Paste screenshot here
(screen listing the stored students)

Conclusion: Thus, the stored student records were retrieved with a Cursor and shown in a ListView, completing the create, insert and read operations of SQLite.

Viva Questions: What is a Cursor and why must it be closed? Difference between rawQuery() and query()? How can SQL injection be avoided (parameterised queries with ?)?

Optional Extras

- Options menu (syllabus assignment 6): create res/menu/main_menu.xml with Settings, Help and Exit items; inflate it in onCreateOptionsMenu() and show a Toast with the item title in onOptionsItemSelected().
- Activity result (syllabus assignment 10): launch SecondActivity with startActivityForResult(), return an entered age with setResult(RESULT_OK, data), and show it in a TextView inside onActivityResult().

Journal Completion

Total practicals completed: ____ / 10 Term work marks: ____ / 10

Teacher's signature: _____ Date: _____